



הרצאה 12 – Full Stack

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

1. Express – public static files
2. ניהול שגיאות
3. SDD
4. המשך תרגיל posts – חיבור צד לקוח אמיתי
5. דטבייס – בדיקת תשובה של יצירת רשומה
6. סיכום קורס



Express – public .1 static files

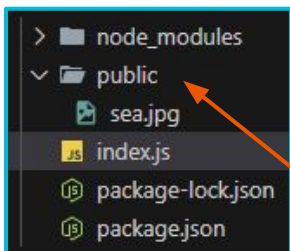
Public (static) files

- 
- `express.static` הוא middleware שתפקידו שליחת קבצים מספריית `public` כך אפשר לגשת ישירות לתמונות ולכל קובץ שנמצא בספריית אלו
 - אקספרס מגיע עם `middleware` מובנה אחד שמחבר בין בקשת `assets` מהלקוח לבין ספריית `public` שבשרת:
`express.static(root, [options])`
 - `root`: חובה, הנתיב לספרייה בשרת ממנה ישלחו הקבצים הסטטיים
 - `Options`: אופציונלי, מכיל כל מיני הגדרות
 - מקובל לייצר ספריית `public` לקבצים כלליים כמו גלריות תמונות, `pdf` וכן הלאה
 - הפרמטר `__dirname` הוא פרמטר גלובלי של `Node.js` והוא מחזיק את הנתיב האבסולוטי של המודול שבו אנו נמצאים (התיקיה הראשית של הפרויקט)

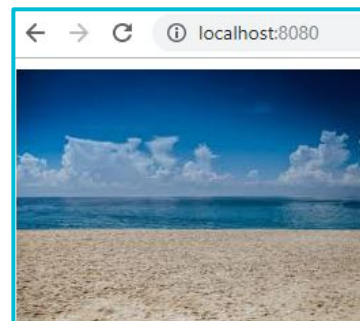
Public (static) files

● דוגמה לשימוש בקובץ סטטי שנמצא על השרת

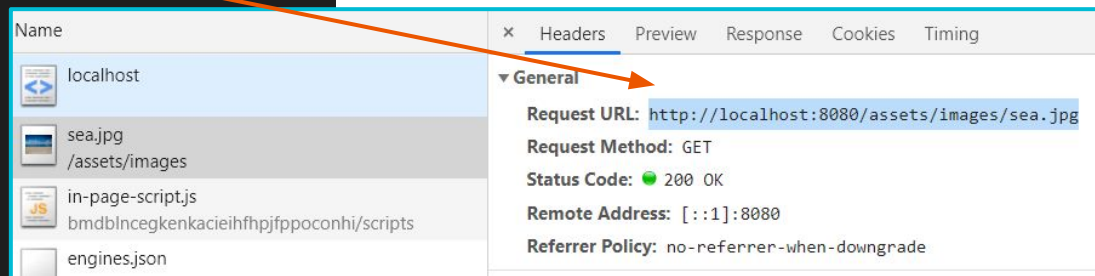
● שימו לב - מבחינת הדפדפן הקבצים מגיעים מספריית assets, לא public



```
JS index.js x
JS index.js ▸ ...
1 const express = require('express');
2 const app = express();
3 const port = process.env.PORT || 8080;
4
5 app.use("/assets", express.static(`${__dirname}/public`)); // Client can't see public
6
7 app.get("/", (req,res) => {
8   res.send("<!doctype html>
9   <html>
10     <head></head>
11     <body>
12       <img src='assets/images/sea.jpg' >
13     </body>
14   </html>'");
15 });
16
17 app.all("/*", (req,res, next) => { // Fallback
18   res.send("Got lost? This is a friendly page :-)");
19 });
20
21 app.listen(port);
22 console.log(`listening on port ${port}`);
```



```
<html>
  <head></head>
  <body cz-shortcut-listen="true">
    
  </body>
</html>
```



Public (static) files

דוגמה לשליחת קובץ index.html מהשרת ללקוח, ע"י `res.sendFile`

The image shows a code editor with two files open: `index.js` and `index.html`. An orange arrow points from the `res.sendFile` call in `index.js` to the `index.html` file. Below the code editor, a file explorer shows the `public` directory containing `index.html` and `index.js`. To the right, a browser window at `localhost:8080` displays the text "I'm back !".

```
JS index.js x
JS index.js ▶ ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/", (req,res) => {
6    res.sendFile(`${__dirname}/public/index.html`);
7  });
8
9  app.listen(port);
10 console.log(`listening on port ${port}`);
```

```
<> index.html x
public ▶ <> index.html ▶ html ▶ head
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title></title>
5    </head>
6    <body>
7      <h1>I'm back !</h1>
8    </body>
9  </html>
```

```
public
  <> index.html
JS index.js
```

localhost:8080

I'm back !



2. ניהול שגיאות

ניהול שגיאות

- עלינו לשים לב מתי לזרוק exception ומתי להחזיר שגיאה.
בשמחזירים שגיאה עלינו להחליט מה הסטטוס קוד שחוזר, זה יכול להיות גם 200, הכל תלוי בהגדרות שלנו וגם במה שצד הלקוח יעשה עם זה
- כשמתכננים API יש לנו אפשרות להחזיר אובייקט הצלחה או אובייקט כשלון, הם לא חייבים להיות עם אותם מאפיינים (שדות)
- דוגמאות:
 - אין נתונים בדטבייס (מגיעה רשימה ריקה) - שגיאה או לא?
 - מחיקה לא בוצעה בדטבייס - איך ננהל?
 - הנתיב לא נתמך בשרת

ניהול שגיאות - המשך



- אפשר לקרוא עוד, למשל:

<https://blog.postman.com/best-practices-for-api-error-handling/>

```
{
  "status": "error", // This can be 'error' or 'success'
  "statusCode": 404,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "The requested resource was not found.",
    "details": "The user with the ID '12345' does not exist in our records.",
    "timestamp": "2023-12-08T12:30:45Z",
    "path": "/api/v1/users/12345",
    "suggestion": "Please check if the user ID is correct or refer to our documentation at https://",
  },
  "requestId": "a1b2c3d4-e5f6-7890-g1h2-i3j4k5l6m7n8",
  "documentation_url": "https://api.example.com/docs/errors"
}
```

ניהול שגיאות - דוגמה

- חפשו - איפה הקוד מטפל בשגיאות ולמה? שימו לב - יש כמה דרכים לממש!

```
JS index.js x
JS index.js ▶ app.get("/jobExperience/jobId") callback
1 const express = require('express');
2 const app = express();
3 const data = require("../data/company.json");
4 const port = process.env.PORT || 8080;
5
6 app.get("/jobExperience/:jobId", (req, res) => {
7   let foundJob = false;
8
9   for (let i in data.jobs) {
10     const job = data.jobs[i];
11
12     if (job.id == req.params.jobId) {
13       console.log(`Found job id: ${req.params.jobId}`);
14
15       foundJob = true;
16       res.status(200).json({"exp-years": job.experience});
17     }
18   }
19
20   if (!foundJob) {
21     res.status(200).json({"error": "Job not found"});
22   }
23 });
24
25 app.listen(port);
26 console.log(`listening on port ${port}`);
```

```
{ } company.json x
data ▶ { } company.json ▶ [ ] jobs ▶ { } 2
1 {
2   "company-name": "D.W",
3   "client-id": 775,
4   "jobs": [
5     {
6       "id": 54,
7       "cat": "se",
8       "subject": "Full stack developer",
9       "experience": 1,
10      "status": "open"
11    },
12    {
13      "id": 57,
14      "cat": "se",
15      "subject": "Front end developer",
16      "experience": 2,
17      "status": "open"
18    },
19    {
20      "id": 86,
21      "cat": "is",
22      "subject": "System designer",
23      "experience": 5,
24      "status": "closed"
25    }
26  ]
27 }
```

localhost:8080/jobExperience/54
{"exp-years":1}

localhost:8080/jobExperience/57
{"exp-years":2}

localhost:8080/jobExperience/55
{"error":"Job not found"}



SDD .3

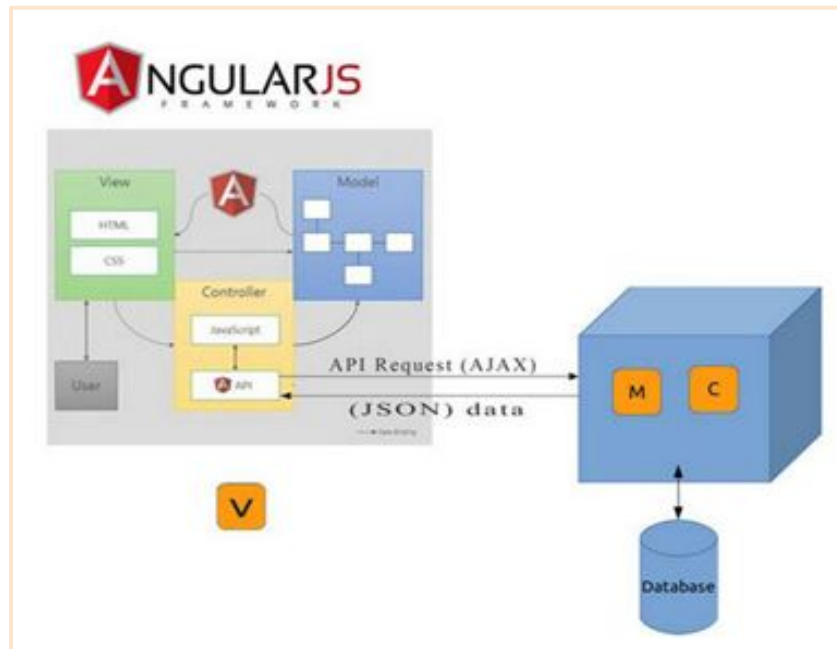
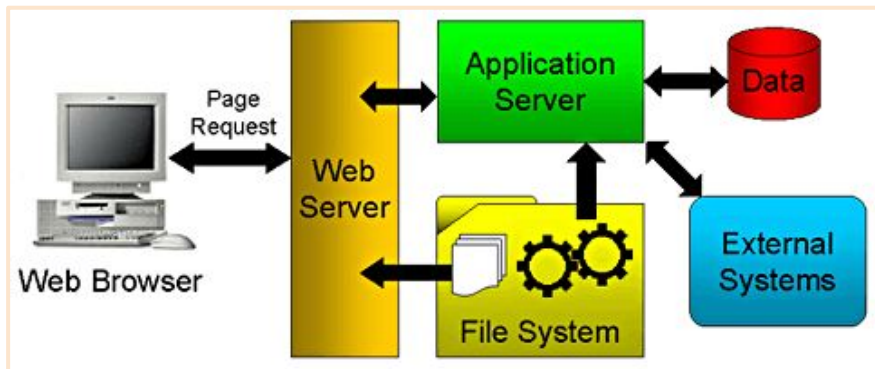
SDD



- SDD = Software Design Document
- מסמך עיצוב תוכנה – תיאור כתוב של מוצר תוכנה כדי לתת הכוונה למפתחים על הארכיטקטורה של התוכנה.
- ארכיטקטורה ו-Design הם היבטים מאוד מרכזיים בפיתוח מערכת תוכנה.
הם עוזרים לחשוב על התרחישים השונים, האובייקטים המרכזיים, המחלקות, מסד הנתונים, האינטראקציות...
- בקורס הנוכחי נממש מיני-SDD, כדי שיעזור להבין את מבנה הפרוייקט, מבנה מסד הנתונים ואת האינטראקציות במערכת. התרשימים שיהיו עליכם לייצר:
 - Database design
 - Use case diagram
 - MVC diagram

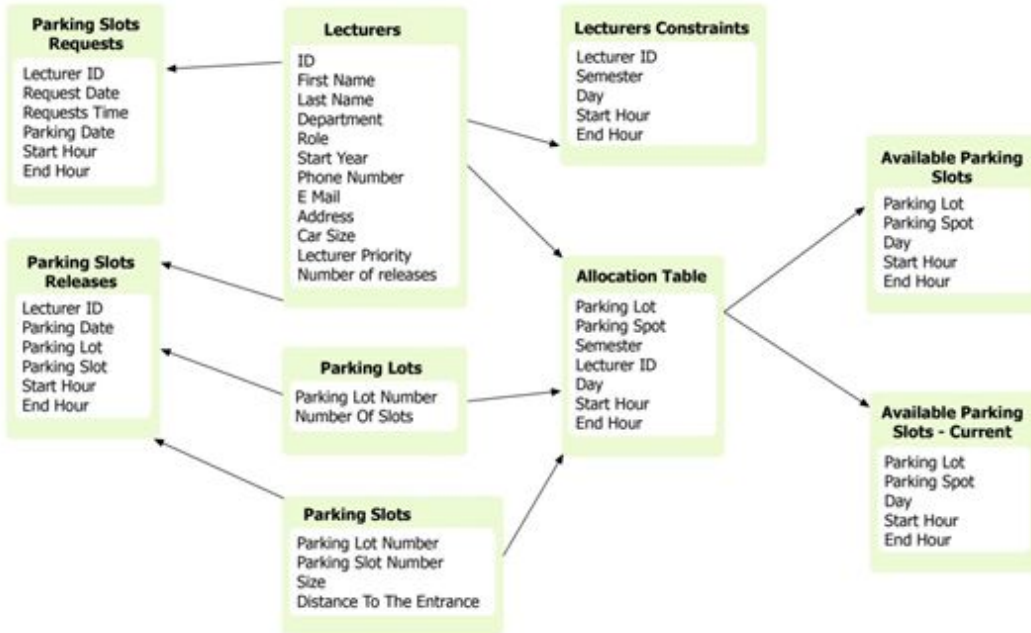
תרשימים

- תרשים צריך לתת מידע על המערכת הספציפית שלנו. אלו דוגמאות לא טובות:



Database design

- תיאור המידע במסד הנתונים שלנו, מכיל ישויות והקשרים ביניהן.
- ישות יכולה להיות אובייקט מוחשי, כמו תלמיד או בית-ספר, או אובייקט אבסטרקטי כמו עיסקה.
- לכל ישות יהיו מאפיינים (מקביל לשדות בטבלה).

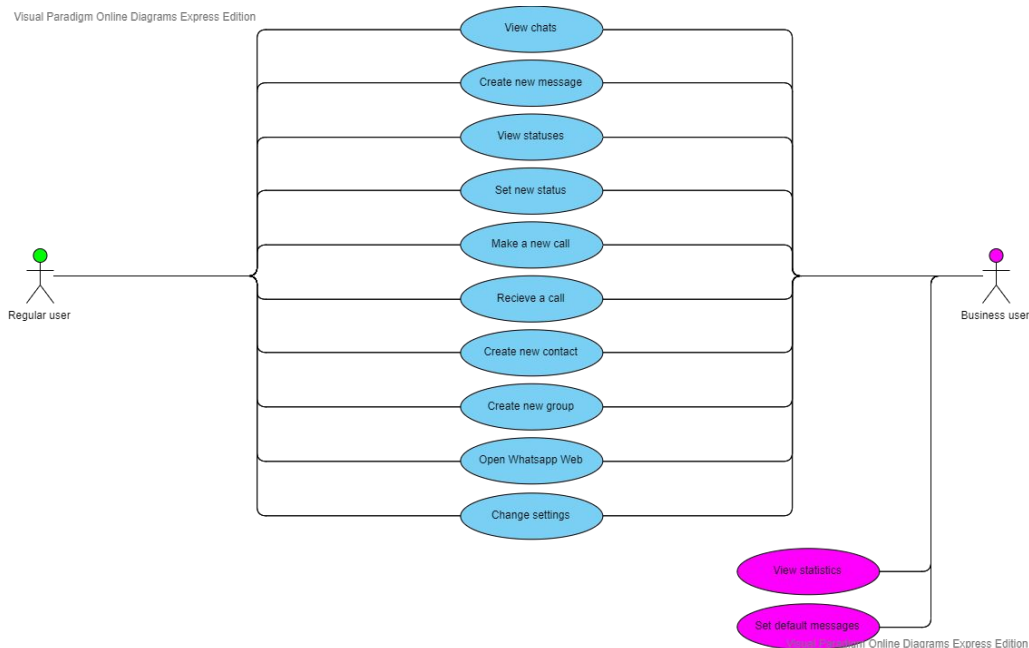


Use case diagram

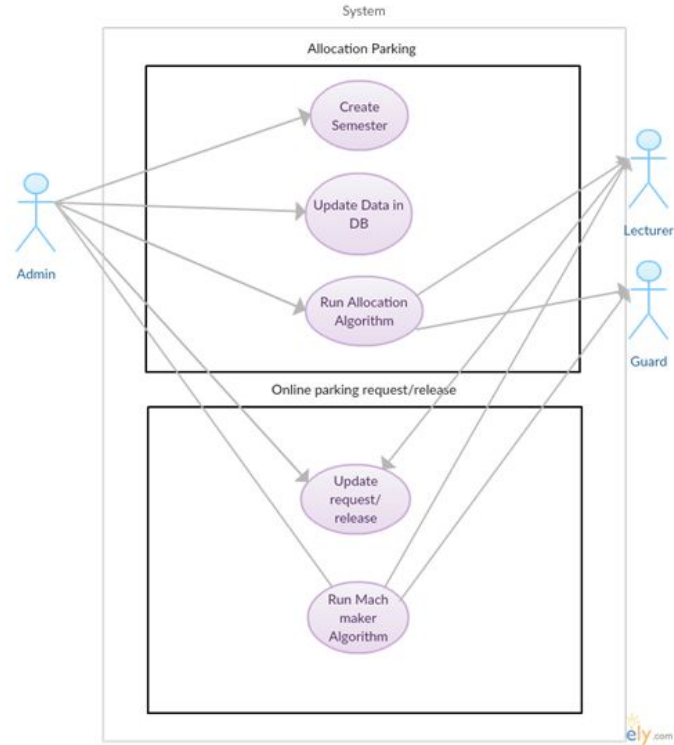
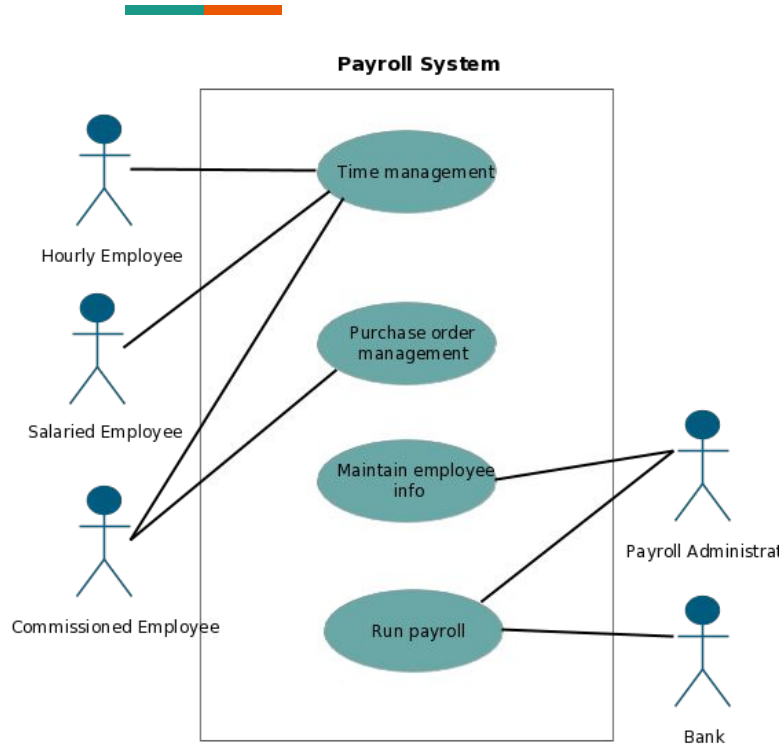
- דיאגרמות של 'מקרי שימוש' (use case) הן דיאגרמות התנהגות המשמשות לתיאור מערך פעולות שמערכת או משתמש יכולים לבצע.
- כל use case אחד אמור לספק תוצאה אחת במערכת.

בדוגמה:

תרשים use cases מרכזיים ב-whatsapp.



Use case diagram

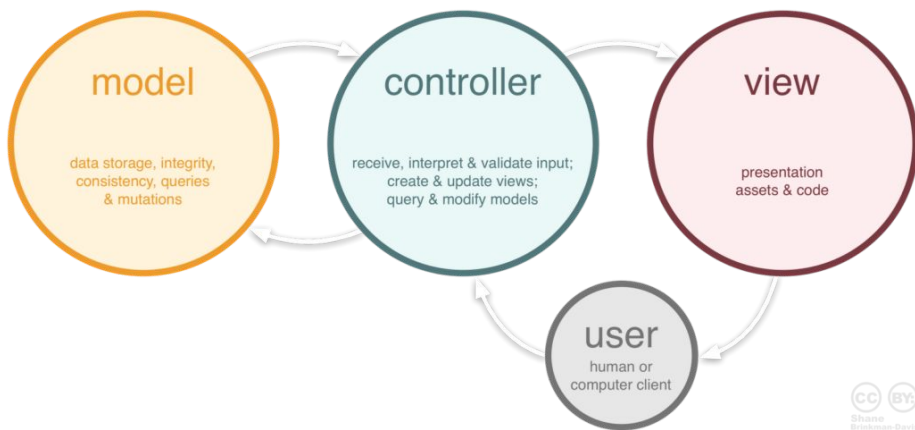


MVC diagram

תרשים MVC מתאר את שכבות המערכת, ה-Model-Controller-View.

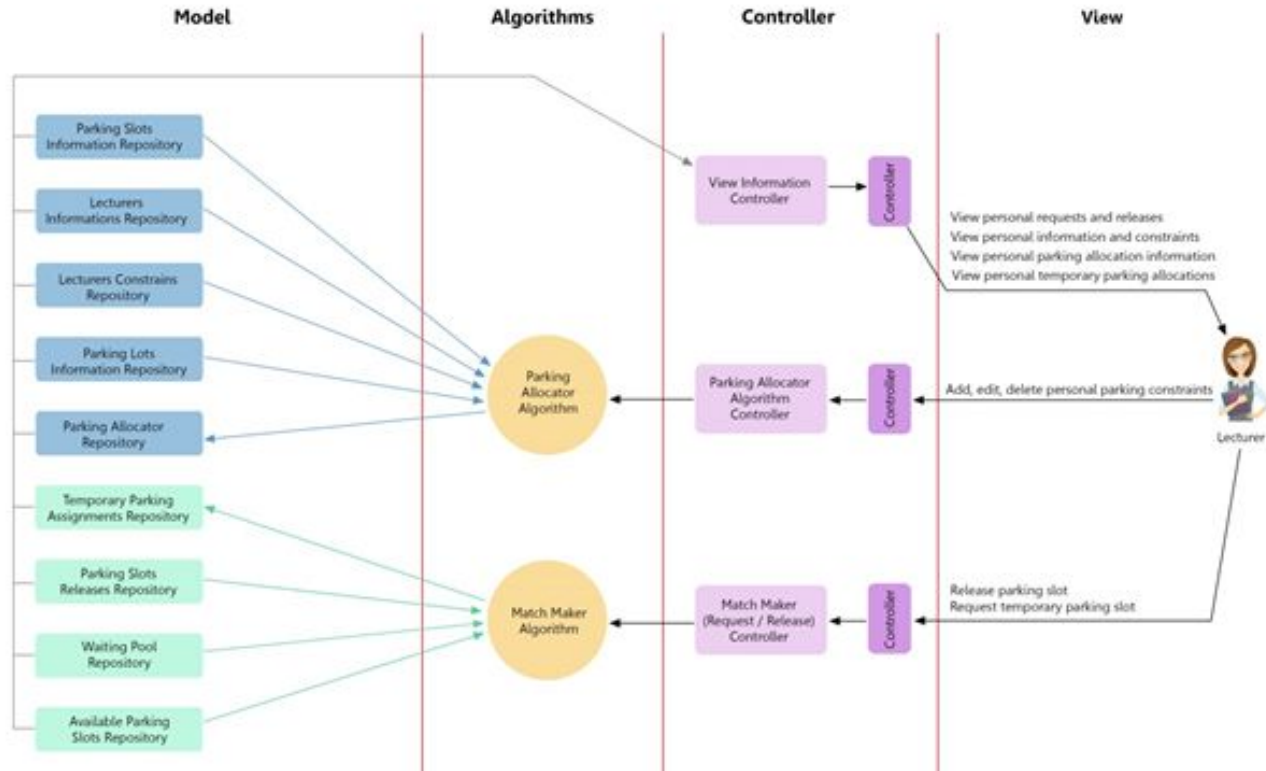
בתרשים רואים את הזרימה בין השכבות:

פעולה מסוימת נעשית ב-view, לאיזה קונטרולר היא מגיעה, לאיזו לוגיקה שייכת ועל איזה ישות ומסד נתונים היא משפיעה.



MVC diagram

Software Architecture Pattern - MVC



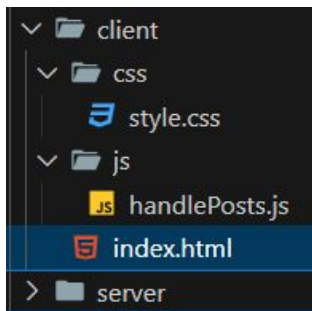


4. המשך תרגיל posts –
חיבור צד לקוח אמיתי

תרגיל posts - צד לקוח - 1. עמוד index.html

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
5     <link rel="stylesheet" href="css/style.css"/>
6     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="
7     <script src="js/handlePosts.js"></script>
8     <title>Posts list</title>
9   </head>
10  <body>
11    <header>
12      <h1>Posts list</h1>
13    </header>
14    <main></main>
15  </body>
16 </html>
```

Posts list



תרגיל posts - צד לקוח - 2. הבאת רשימת posts

Posts list

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

```
X Headers Preview Response Initiator Timing
▼ [{"id": 11, "title": "Default post title", "description": "Default post desc"},...]
  ▶ 0: {"id": 11, "title": "Default post title", "description": "Default post desc"}
  ▶ 1: {"id": 12, "title": "Default post title", "description": "Default post desc"}
  ▶ 2: {"id": 15, "title": "Default post title", "description": "Default post desc"}
  ▶ 3: {"id": 18, "title": "Default post title", "description": "Default post desc"}
  ▶ 4: {"id": 22, "title": "Default post title", "description": "Default post desc"}
  ▶ 5: {"id": 23, "title": "Default post title", "description": "Default post desc"}
```

X	Headers	Preview	Response	Initiator	Timing
▼ General					
Request URL:		http://127.0.0.1:8081/api/posts			
Request Method:		GET			
Status Code:		200 OK			
Remote Address:		127.0.0.1:8081			
Referrer Policy:		strict-origin-when-cross-origin			
▼ Response Headers		☐ Raw			
Access-Control-Allow-Headers:		Origin, X-Requested-With, Content-Type, Accept			
Access-Control-Allow-Methods:		GET, POST, PUT, DELETE			
Access-Control-Allow-Origin:		*			
Content-Length:		439			
Content-Type:		application/json; charset=utf-8			
Date:		Thu, 27 Jun 2024 17:05:23 GMT			
Etag:		W/"1b7-0X/TICEToFJR/ZytAvv5+qSaWc0"			
X-Powered-By:		Express			
▶ Request Headers (15)					

תרגיל posts - צד לקוח - 2. הבאת רשימת posts

```
1 function createPostItem(postData) {  
2   const post = document.createElement('li');  
3   post.classList.add("list-group-item");  
4  
5   const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;   
6   post.innerHTML = postListItem;  
7  
8   return post;  
9 }
```

```
10  
11 > function populatePostsList(postsData) { ...  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23 function getPostsListFromServer() {  
24   fetch("http://127.0.0.1:8081/api/posts")  
25     .then(response => response.json())  
26     .then(postsData => populatePostsList(postsData));  
27 }  
28  
29 window.onload = () => {  
30   getPostsListFromServer();  
31 }
```

```
function populatePostsList(postsData) {  
  const postsList = document.createElement('ul');  
  postsList.classList.add("list-group");  
  
  for (const i in postsData) {  
    const post = createPostItem(postsData[i]);  
    postsList.appendChild(post);  
  }  
  
  document.getElementsByTagName("main")[0].appendChild(postsList);  
}
```

תרגיל posts - צד לקוח - 3. הוספת post

Add a post

Posts list

[Default post title - Default post desc](#)

[Default post title - Default post desc](#)

Name	✕ Headers	Payload	Preview	Response	Initiator	Timing
posts	▼ General					
Request URL:		http://127.0.0.1:8081/api/posts				
Request Method:		POST				
Status Code:		● 200 OK				
Remote Address:		127.0.0.1:8081				
Referrer Policy:		strict-origin-when-cross-origin				

✕ Headers	Payload	Preview	Response	Initiator	Timing
▼ Request Payload view source					
▼ {title: "Default post title", description: "Default post desc"} description: "Default post desc" title: "Default post title"					

✕ Headers	Payload	Preview	Response	Initiator	Timing
1 true					

תרגיל posts - צד לקוח - 3. הוספת post

```
29 function addCreatedPostToList(response, defaultPostBody) {
30   if (response.status !== 200) {
31     return;
32   }
33
34   const post = createPostItem(defaultPostBody);
35   const postsList = document.querySelector("main > ul");
36   postsList.appendChild(post);
37 }
38
39 function addDefaultPost() {
40   const defaultPostBody = {
41     "title": "Default post title",
42     "description": "Default post desc"
43   }
44
45   fetch("http://127.0.0.1:8081/api/posts", {
46     method: "POST",
47     headers: {
48       "Content-Type": "application/json",
49     },
50     body: JSON.stringify(defaultPostBody)
51   })
52   .then(response => addCreatedPostToList(response, defaultPostBody));
53 }
54
55 function addListeners() {
56   document.getElementById("add-post-button").onclick = () => addDefaultPost();
57 }
58
59 window.onload = () => {
60   getPostsListFromServer();
61   addListeners();
62 }
```


The screenshot shows the Chrome DevTools Network tab. A list of network requests is on the left, with the first one (ID 26) selected. The main panel shows the details for this request. The 'General' tab is active, displaying the Request URL as 'http://127.0.0.1:8081/api/posts/26', the Request Method as 'DELETE', the Status Code as '200 OK' (indicated by a green dot), the Remote Address as '127.0.0.1:8081', and the Referrer Policy as 'strict-origin-when-cross-origin'. Below the 'General' tab, there are expandable sections for 'Response Headers (10)' and 'Request Headers (15)'.

תרגיל posts - צד לקוח - 4. מחיקת פוסט

```
1 function deletePostItemFromList(response, postId) {
2   if (response.status !== 200) {
3     return;
4   }
5
6   const postItemToDelete = document.querySelectorAll(`a[href='post.html?postId=${postId}']`);
7   postItemToDelete[0].parentElement.remove();
8 }
9
10 function deletePost(postId) {
11   fetch(`http://127.0.0.1:8081/api/posts/${postId}`, {
12     method: "DELETE",
13     headers: {
14       "Content-Type": "application/json",
15     }
16   })
17   .then(response => deletePostItemFromList(response, postId));
18 }
19
20 function createPostItem(postData) {
21   const post = document.createElement('li');
22   post.classList.add("list-group-item");
23
24   const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;
25   post.innerHTML = postListItem;
26
27   const deletePostButton = document.createElement('button');
28   deletePostButton.classList.add("btn", "btn-danger");
29   deletePostButton.setAttribute("id", "delete-post-button");
30   deletePostButton.setAttribute("type", "button");
31   deletePostButton.textContent = "delete";
32   deletePostButton.addEventListener("click", () => deletePost(postData.id));
33
34   post.appendChild(deletePostButton);
35
36   return post;
37 }
```



5. דטבייס – בדיקת התשובה של יצירת רשומה

דטבייס - בדיקת תשובה של יצירת רשומה

- בהרצאה הקודמת ראינו את ההכנסה, העדכון והמחיקה בדטבייס, אבל לא בדקנו את התשובה, החזרנו ישר בוליאני וסיכמנו שנחזור לזה. בדוגמה הבאה אנחנו בודקים את התשובה.
- שימו לב - יש כאן ערבוב של `async await` עם `promises` כדי להדגים שזה אותו הדבר.
כשאתם תתכנתו - תבחרו שיטה אחת, עדיף `async await`, ושימו לב שלא תהיינה הרבה רמות של אסינכרוניות בקוד, כמו `then` 5 אחד בתוך השני

```
22 // POST localhost:8081/posts
23 async addPost(req, res) {
24   const { dbConnection } = require('../db_connection');
25   const connection = await dbConnection.createConnection();
26
27   const { body } = req;
28   await connection
29     .execute(`INSERT INTO tbl_99_posts (title, description) VALUES ("${body.title}", "${body.description}")`)
30     .then((response) => {
31       console.log(response);
32       connection.end();
33       res.send(response[0].affectedRows === 1);
34     });
35 },
```

דטבייס – החזרת id לאחר יצירת רשומה

- בהוספה, נצטרך גם להחזיר את ה-id של האלמנט החדש שנוצר, אחרת תהיינה פעולות שלא נוכל לבצע, כמו הבאת האובייקט הזה או מחיקתו.

```
[
  ResultSetHeader {
    fieldCount: 0,
    affectedRows: 1,
    insertId: 42,
    info: '',
    serverStatus: 2,
    warningStatus: 0,
    changedRows: 0
  },
```

```
22 // POST localhost:8081/posts
23 async addPost(req, res) {
24   const { dbConnection } = require('../db_connection');
25   const connection = await dbConnection.createConnection();
26   const { body } = req;
27
28   await connection
29     .execute(`INSERT INTO tbl_99_posts (title, description) VALUES ("${body.title}", "${body.description}")`)
30     .then(response => {
31       if (response[0].affectedRows === 1) {
32         getPostById(response[0].insertId)
33           .then(row => res.json(row));
34       }
35       else {
36         // ?
37       }
38     });
39 },
```

דטבייס - החזרת id לאחר יצירת רשומה - המשך

- כתבנו פונקציה שתביא את הפוסט לפי ה-id, איך אפשר לייעל את הקוד?

```
63 async function getPostById(postId){
64     const { dbConnection } = require('../db_connection');
65     const connection = await dbConnection.createConnection();
66
67     const [rows] = await connection.execute(`SELECT * from tbl_99_posts WHERE id=${postId}`);
68
69     connection.end();
70     return rows[0];
71 }
```

Name	✕ Headers Payload Preview Response Initiator Timing
posts	▼ {id: 55, title: "Default post title", description: "Default post desc"} description: "Default post desc" id: 55 title: "Default post title"

דטבייס - החזרת id לאחר יצירת רשומה - המשך

● נחזור לצד הלקוח

```
57 function addCreatedPostToList(response) {
58   const post = createPostItem(response);
59   const postsList = document.querySelector("main > ul");
60   postsList.appendChild(post);
61 }
62
63 function addDefaultPost() {
64   const defaultPostBody = {
65     "title": "Default post title",
66     "description": "Default post desc"
67   }
68
69   fetch("http://127.0.0.1:8081/api/posts", {
70     method: "POST",
71     headers: {
72       "Content-Type": "application/json",
73     },
74     body: JSON.stringify(defaultPostBody)
75   })
76   .then(response => response.json())
77   .then(data => addCreatedPostToList(data));
78 }
```

```
<ul class="list-group"> flex
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> == $0
    <a href="post.html?postId=62">Default post title - Default post desc</a>
    <button class="btn btn-danger" id="delete-post-button" type="button">delete
    </button>
  </li>
  > <li class="list-group-item">
    <a href="post.html?postId=63">Default post title - Default post desc</a>
    <button class="btn btn-danger" id="delete-post-button" type="button">delete
    </button>
  </li>
</ul>
```

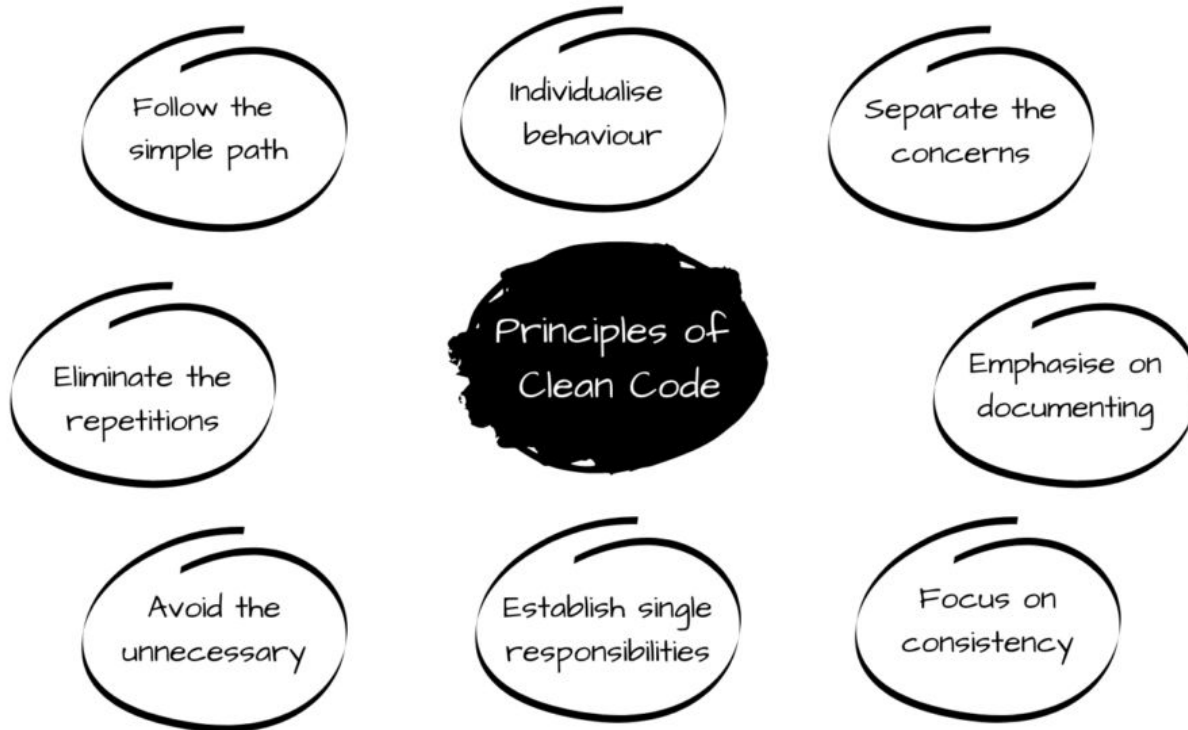


6. סיכום קורס

חלק ממה שלמדנו הסמסטר



Clean code



THE PRINCIPLES OF CLEAN CODE

- C**hoose descriptive, meaningful names
- L**ean on tests to guide your development
- E**nsure your code is self-explanatory
- A**void large functions
- N**ever be redundant, keep it simple

Soft skills



THE IMPORTANCE OF SOFT SKILLS FOR SOFTWARE DEVELOPERS

- 1 Effective Communication
- 2 Team Collaboration
- 3 Adaptability in a Dynamic Environment
- 4 Problem-Solving
- 5 Client Interaction
- 6 Time Management